Lab 5: Static Code Analysis

Student: Pruthviraj H

SRN: PES2UG24CS381

Program: pharmacy_system.py

This report documents the use of Pylint, Bandit, and Flake8 to improve the quality, security, and style of a small pharmacy stock tracker.

Issue Table

Issue	Type	Line(s)	Description	Fix approach
SQL injection risk	Security	21	User input was joined directly into an SQL query.	Used a parameterized query.
Mutable default argument	Bug	24	A default list was shared across calls.	Used None and created a new list.
Bare exceptions	Reliability	38, 58	Errors were hidden by bare except blocks.	Caught only expected exception types.
Unsafe file handling	Quality	50, 57	Files lacked context managers and encoding.	Used with blocks and UTF-8 encoding.
Style and unused import	Style	3, multiple	Unused os import and naming/spacing issues.	Removed, renamed, and formatted code.

Final Results

Tool	Result
Pylint	10.00/10
Bandit	No issues identified
Flake8	No issues identified

Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest issues were the unused import, spacing, naming, and line-length warnings because they only required small formatting or cleanup changes. The hardest issue was the SQL injection risk because it required understanding parameterized SQL and changing the query without changing the program's result.

2. Did the static analysis tools report any false positives? If so, describe one example.

There were no important false positives in this analysis. The Bandit SQL warning was reported with low confidence, but it was still a valid concern because the code directly inserted a user-provided name into the SQL query. The fix improved the code even though the sample inputs were harmless.

3. How would you integrate static analysis tools into your actual software development workflow?

I would run Flake8 and Pylint locally before committing code, and run Bandit as part of the continuous integration pipeline. The pipeline could fail for high-severity security findings and important style or quality errors, while still allowing review of lower-priority warnings.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

The code became easier to read through clear function names, docstrings, consistent formatting, and context managers. It became more robust through specific exceptions and removal of the mutable default argument, and safer through the parameterized query. Pylint improved from 6.90/10 to 10.00/10, with no Bandit or Flake8 findings.

Analyzer Output

pylint_report.txt

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

bandit_report.txt

```
Run started:2026-10-16 14:40:50.093592+00:00

Test results:
■No issues identified.

Code scanned:
■Total lines of code: 97
■Total lines skipped (#nosec): 0
■Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
■Total issues (by severity):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
■Total issues (by confidence):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
Files skipped (0):
```

flake8_report.txt

```
No findings.
```